

Build Plan — 2026-06-03 (Revised)

mini-moi personal AI agent platform For review: OpenClaw + Grok — validate sequencing, flag conflicts, surface risks.

Framing (what changed from the first draft)

Two tracks, two branches, two risk classes:

- **Daily-use improvements** → `main`. Small, reversible, ship today.
- **Database spine (Guild platform setup)** → `guild` branch. Additive, optional, **does not merge today**. The goal is to **prove the concept**, not to put a database into daily use.

The spine is **official Guild platform infrastructure** (per the Guild charter: the Postgres + graph spine is shared platform, Guild-governed, Curator is its first consumer). It is framed and built as early Guild setup, not as Curator backend.

Three governing principles for the spine

1. **Daily use never depends on the database.** The system runs on JSON flat files today and must keep running on JSON if Postgres/Neo4j are down, broken, or absent. The DB is strictly additive and strictly optional. Nothing on a daily read path may depend on it.
2. **JSON is the source of truth; the DB is a rebuildable projection.** The flat files remain authoritative. Postgres/Neo4j are a view of them that can be dropped and regenerated from JSON at any time. This makes schema change safe — alter schema, re-run migrate, done. No future agent treats the DB as authoritative.
3. **First DB try → keep it limited.** Prove the spine works and the graph traverses real research data. Do not migrate everything. Smaller surface, less to go wrong.

Spend follows attention

No automatic AI calls, no unattended pipeline steps. Applies to build too: no speculative abstraction, no over-engineering. The schema is expected to evolve — build the minimum that proves the POC.

Context — where we are

- B-020 (German Test 8) quarantined; B-021 (Taiwan-relevance) demoted to non-gating ENV_CHECK.
- Baseline: German 48/48 (1 skipped). Curator UAT 5/5 gating pass.

- `main` clean, up to date with origin.
 - Docker installed, not running. `docker-compose.yml` updated (Neo4j 512m, Postgres 256m, `restart: no`). Images not pulled.
-

TRACK A — Daily-use improvements (branch: `main`)

Small, reversible, real value, zero DB risk. If the day ends here, it was a good day.

Phase 1 — German core loop (do 1b FIRST)

1b. Full Grok Voice prompt delivery (HIGHEST VALUE — do first) (~30 min)

Files: `domains/german/html_server.py`, `domains/german/templates/german_gesprache.html` **Problem:** `getPromptText()` returns only the brief scene stub, not the full Grok Voice system prompt. Copy/Save deliver a stub a user can't actually run a session from. **Fix:** `POST /api/gesprache-prompt` accepting `{persona_name, scene_key}`; calls existing `build_persona_prompt()` in `german_domain.py`; returns the full assembled prompt. UI calls it on scene selection, populates `#prompt-text`. **Why first:** This is the core loop. Everything else in Phase 1 is polish; this is function.

1a. Landing page stale link (~5 min)

File: `domains/german/templates/german_landing.html` `/ueben` → `/gesprache`, label "Üben" → "Gespräche". (301 works; this removes the rough edge on the public entry point.)

1c. Scene button German labels (~10 min)

Files: `domains/german/data/config/personas/*.json` Add a `labels` map per persona (`"cafe_order": "Ich hätte gerne..."`). JS reads `p.labels?.[key] || key.replace(/_/g, '')`. No backend change.

→ COMMIT + STOP-GATE 1. Clean place to end the day.

Phase 2 — Curator daily-pipeline gaps

2a. Subnav → Research link (~10 min)

Add a `Research` link to the Curator subnav (currently no link to `/research/dashboard`). Simple HTML.

2b. Dormant section routing (~20 min) — reads JSON, stays JSON

File: `curator_rss_v2.py` After the top-20 score-ranked selection (unchanged), a second pass checks remaining candidates against active Topic tags via **deterministic string-match only, no**

AI call. Matches surface in a collapsed "on radar" section below the main feed, ~10 items, score-ordered. One item, one place (top-20 wins; not duplicated below). **Reads from:** `_NewDomains/research-intelligence/data/threads/*/thread.json` (JSON). **Stays on JSON permanently** — this is a daily-production read path, and per principle #1 it must not depend on the DB. Do **not** re-point it to Postgres in this build or later without a deliberate decision that designs in JSON fallback. **Design ref:** CURATOR_DESIGN_REFERENCE.md, "Dormant section + radar routing" (Hybrid C, settled).

→ COMMIT + STOP-GATE 2. Daily use improved, zero DB risk taken. Track A complete.

TRACK B — Database spine POC (branch: `guild`, does not merge today)

Official Guild platform setup. Additive, optional, prove-the-concept. JSON stays source of truth. Lives on the `guild` branch and stays there — no merge today. The bridge runs additively from the branch so the concept can be validated over several days before anything touches `main`.

Create branch first: `git checkout -b guild` (off `main`, after Track A committed).

Learning note — what is Neo4j even for? (read before 3e)

Robert wants to understand what the graph buys before committing to it. The honest POC framing:

- **Postgres** answers questions about *records you declared*: "list active Topics and their Source counts." Tables and joins. This is the safe, well-understood half.
- **Neo4j** is meant to answer questions about *connections you did NOT declare*: "Topics X and Y are two hops apart through gold-geopolitics — you never linked them directly." Traversal across declared edges to surface latent structure.
- **The POC's job is to test whether that's actually useful on YOUR data, at YOUR current scale.** It may well show that with only 6 topics and a handful of sources, the graph surfaces nothing a human couldn't see at a glance — in which case the honest finding is "Neo4j earns its place later, when structure is richer," and that is a *successful* POC outcome, not a failure. The point is to learn, cheaply, whether and when the graph pays off. Do not assume it must justify itself today.

3a. Docker up + image pull (~15 min)

```
bash
```

```
open -a Docker
docker compose pull      # postgres:16, neo4j:5 (~500MB)
docker compose up -d
docker compose ps        # both healthy
```

Postgres: 5432, db `personal_agents`, user `minimoi`, pw in keychain. Neo4j: 7687 (Bolt) / 7474 (browser), pw in keychain.

3b. PostgreSQL schema — research objects ONLY (~20 min)

New file: `db/schema.sql` Six tables, all of which feed the graph: `sources`, `topics`, `groups`, `leanings`, `evidence`, `tag_aliases`. (Schemas as in the original plan.) **Dropped from today:** `german_sessions` and `curator_signals` — they do not participate in the graph, they are the riskiest migration (415+ sessions with schema drift), and copying them in is scope creep. Defer to a later migration when something actually reads them from Postgres. **Schema is expected to evolve** — build the minimum, expect to ALTER. JSON regenerates it.

3c. Migration JSON → Postgres (~30 min) — idempotent

New file: `db/migrate.py` Upsert-on-primary-key (re-runnable). Order:

1. `tag_aliases.json` → `tag_aliases`
2. `threads/*/thread.json` → `topics` (6)
3. `sources/sources.json` → `sources`
4. `groups/groups.json` → `groups`
5. `leanings/leanings.json` → `leanings` + `evidence` Verify: row counts match JSON record counts; log skipped/failed. **Reminder:** this reads JSON (the source of truth) and projects into Postgres. Re-runnable any time, including after a schema change.

3d. Postgres CRUD layer (~30 min)

New file: `db/postgres.py` — connection pool (`psycopg2/psycopg3`, not `SQLAlchemy` — see Q1), CRUD for the six tables. Pure data layer, tested standalone, no Flask wiring.

3e. Neo4j graph seed (~30 min)

New file: `db/graph_seed.py` — reads from Postgres (not JSON), writes declared edges. MERGE not CREATE (idempotent). Nodes: Topic, Source, Group, Leaning, Tag. Edges: MEMBER_OF, LINKED_TO, TAGGED, ATTACHED_TO, SUPPORTED_BY{stance}, ALIAS_OF. (As original plan.)

3f. Neo4j driver layer (~20 min)

New file: `db/neo4j_driver.py` — official `neo4j` Bolt driver (see Q2).

```
get_topic_neighbors(slug, hops=2), get_sources_for_leaning(id),  
get_related_topics(slug)).
```

3g. POC verification (~20 min)

New file: `db/poc_verify.py` — prints results for:

1. Postgres join: active Topics + Source counts.
2. Neo4j traversal: Topics within 2 hops of `gold-geopolitics`.
3. Cross-system: Sources supporting "Hungry and Poland" (Postgres) → other Topics those Sources connect to (Neo4j). Pass = both DBs return data, no errors, results sensible. **Also capture the honest read: did the graph surface anything non-obvious? Record the answer** — that's the actual learning output.

→ COMMIT to `guild` (STOP-GATE 3). Spine proven on the branch. Clean place to end the day.

Phase 4 — Dual-write bridge + drift check (~1 hr)

New file: `db/bridge.py` — thin wrapper. Existing domain functions call `bridge.write(object_type, data)` after their JSON write. Bridge does Postgres upsert + Neo4j edge update if relevant.

- **Failure mode: log-and-continue.** JSON write already happened; production never breaks. (Answers Q3, with the addition below.)
- **New file:** `db/reconcile.py` — compares JSON record counts to Postgres row counts, reports drift. Run at end of day and after any bridge change. This is what makes log-and-continue safe: drift is *detected*, not silently accumulated.
- **Wiring (on `guild` branch only):** `tools/curator_research.py` and `domains/german/german_domain.py` call the bridge after their JSON writes.
- **Untouched:** `curator_rss_v2.py` scoring, Flask routes, Telegram, launchd.

→ COMMIT to `guild` (STOP-GATE 4). Bridge runs additively from the branch. Validate over several days before considering merge.

What is NOT in scope today

- Merging `guild` to `main` (prove the concept first; merge is a later, deliberate decision)
- German sessions + curator_signals migration (deferred — not graph participants)
- Postgres/Neo4j as a read source for ANYTHING (additive only; daily use stays on JSON)
- 2b reading from Postgres (stays JSON, permanently, per principle #1)

- Leaning UI, dormant section UI beyond 2b routing, Admin/Archiv tabs, persona memory
 - Docker in production / launchd (local dev only)
-

Definition of done

Track A (`main`) — the must-have:

- ☐ 1b: full Grok Voice prompt delivered (core loop works)
- ☐ 1a, 1c: landing link fixed, scene labels German
- ☐ 2a: Research link in Curator subnav
- ☐ 2b: dormant routing active (reads JSON), one-item-one-place respected
- ☐ German suite 48/48 (1 skip); Curator UAT 5/5 gating
- ☐ Committed to `main`

Track B (`guild` branch) — the prove-the-concept, if time:

- ☐ `docker compose up -d` clean, memory caps respected
- ☐ `db/schema.sql` applied (6 research tables)
- ☐ `db/migrate.py` clean, row counts match JSON
- ☐ `db/poc_verify.py` returns sensible results from both DBs — **and the honest read on whether the graph showed anything useful is recorded**
- ☐ `db/bridge.py` + `db/reconcile.py` wired on `guild`; reconcile shows zero drift
- ☐ Committed to `guild`. **Not merged.**

Priority if the day runs short: Track A complete > a half-finished spine. Track A is the guaranteed-good outcome; Track B is the bigger prize but safely bracketed on a branch.

Open questions — resolved

1. **psycopg2 vs asyncpg vs SQLAlchemy** → psycopg2/psycopg3 directly. Flask is sync; no async need; SQLAlchemy ORM is unneeded overhead given dict-shaped data.
2. **neo4j vs py2neo** → official `neo4j` Bolt driver. py2neo lags.
3. **Bridge failure mode** → log-and-continue (never break production) + `reconcile.py` **drift check** (so silent drift can't accumulate). This is the safe-and-honest combination.
4. **German sessions migration scope** → **not today**. Not a graph participant; the 415-session schema drift is a problem to skip, not solve, today.
5. **2b read source** → **JSON, permanently**. It's a daily production read path; per principle #1 it must not depend on the DB.

Files

main:

domains/german/html_server.py	← 1b
domains/german/templates/german_gesprache.html	← 1b JS
domains/german/templates/german_landing.html	← 1a
domains/german/data/config/personas/*.json	← 1c
curator_rss_v2.py	← 2b (reads JSON)
+ Curator subnav HTML	← 2a

guild **branch (NEW** **db/** **subsystem — Guild platform):**

db/schema.sql	db/migrate.py	db/postgres.py
db/neo4j_driver.py	db/graph_seed.py	db/bridge.py
db/reconcile.py	db/poc_verify.py	
tools/curator_research.py		← bridge wiring (guild only)
domains/german/german_domain.py		← bridge wiring (guild only)
docker-compose.yml		← already updated

For OpenClaw + Grok review and Robert sign-off before build starts.